Q-PNA: Quantum-Native p-Adic Neural Architecture — Research Specification v2.0

Author: Rowan Brad Quni-Gudzinas
ORCID: 0009-0002-4317-5604
DOI: 10.5281/zenodo.20287742
Date: 2026-05-19

Abstract

>

Q-PNA: Quantum-Native p-Adic Neural Architecture

Research Specification v2.0

Author: Rowan Brad Quni-Gudzinas **ORCID:** 0009-0002-4317-5604 **DOI:** 10.5281/zenodo.20287742 **Repository:** github.com/QNFO/Q-PNA **Date**: 2026-05-19

Abstract: Q-PNA is a neural network architecture that replaces the continuous embedding spaces of standard deep learning with ultrametric geometry on Bruhat-Tits trees. The architecture consists of four integrated components: p-adic valuation encoding, ultrametric attention with zero learned parameters, tree-walk optimization (a discrete analog of backpropagation), and a syntactic token calculus providing formal verification of every decision. The architecture shares its mathematical foundation with the QWAV quantum computing framework, enabling a unified computing paradigm where the geometry that passively suppresses quantum errors also produces glass-box AI decisions with fully traceable audit trails. This research specification is grounded in prior work including a working proof-of-concept demonstration, a book-length token calculus treatment, and computationally verified cophenetic distance theory, but the full architecture has not yet been computationally validated.

1. Motivation: The Gauge Problem in Continuous AI

1.1 The Archimedean Assumption

Contemporary AI rests on an invisible mathematical assumption: that the space of representations is continuous, smooth, and Archimedean (the real numbers $\mathbb{R}^n$). Every gradient descent step, every backpropagated weight update, every attention score in a transformer, and every latent vector in a variational autoencoder operates within the familiar continuum `[ARCHIVE: Ultrametric Intelligence]`.

This foundation has carried the field remarkably far. Yet it imposes structural limitations that are consequences of the geometry, not of insufficient scale or data:

1.2 Gauge Dependence

`[ARCHIVE: Auditable Attention PoC]`

Embeddings in $\mathbb{R}^d$ are **gauge-dependent**: if $\phi : \mathbb{R}^d \to \mathbb{R}^d$ is any diffeomorphism, the embedding $v \in \mathbb{R}^d$ for concept C can be replaced by $\phi(v)$ without changing intrinsic meaning. Yet neural networks treat these as distinct inputs. Formally:

Definition (Gauge dependence): A representation scheme $R : \mathcal{C} \to \mathbb{R}^d$ mapping concepts to vectors is gauge-dependent if there exists a diffeomorphism ϕ such that training converges to different parameters when trained on $R(c)$ vs. $\phi(R(c))$, even though ϕ preserves all relational structure.

This leads to: - **Adversarial fragility:** Infinitesimal perturbations exploit the continuous corridors in the decision landscape - **Opaque decision-making:** The path from input to output is a composition of nonlinearities with no geometric interpretation - **Poor analogical reasoning:** Similarity in $\mathbb{R}^d$ (cosine, Euclidean) does not correspond to hierarchical similarity

1.3 The Ultrametric Alternative

Ultrametric spaces replace the Archimedean triangle inequality:

$$d(x, z) \leq d(x, y) + d(y, z) \quad \text{(Archimedean)}$$

with the strong triangle inequality:

$$d(x, z) \leq \max(d(x, y), d(y, z)) \quad \text{(Ultrametric)}$$

In an ultrametric space, every triangle is isosceles with the two longest sides equal — a property called **triadic rigidity** `[DOI: 10.5281/zenodo.20213043]`. All balls are clopen (both open and closed). Points are organized in a strict nested hierarchy rather than a continuous manifold.

The natural geometry of ultrametric spaces is a **rooted tree**. The Bruhat–Tits tree $\mathcal{T}_p$ for prime p is the universal ultrametric space: an infinite regular tree with branching factor $p+1$, where the ultrametric distance between two leaves is the p-adic valuation of their difference.

---

2. Mathematical Foundations

2.1 The Bruhat–Tits Tree $\mathcal{T}_p$

`[QWAV-INTERNAL: Technical Deep-Dive §4]`

For a prime p, the Bruhat–Tits tree $\mathcal{T}_p$ is the infinite regular tree where each vertex has exactly $p+1$ neighbors. The boundary of $\mathcal{T}_p$ is the projective line over the p-adic numbers $\mathbb{P}^1(\mathbb{Q}_p)$.

Key properties: - **Regularity:** Every vertex has degree $p+1$ - **Ultrametric:** Distances satisfy the strong triangle inequality - **Group action:** $\mathrm{PGL}(2,\mathbb{Q}_p)$ acts transitively on the vertices - **Langlands connection:** The tree encodes representations of the p-adic group

For computation, we truncate $\mathcal{T}_p$ to finite depth D:

Parameter	Symbol	Typical Value	Role
Prime	p	3	Branching factor $p+1=4$ (ternary tree)
Depth	D	5–10	Number of levels
Leaf count	L	$(p+1)\cdot p^{D-1}$	For $p=3$, $D=5$: $4\cdot 3^4 = 324$ leaves
Semantic primes	k	5–12	Number of prime dimensions for encoding

2.2 Cophenetic Distance

`[DOI: 10.5281/zenodo.20213043]`

For leaves a, b in a rooted tree with height function h, the **cophenetic distance** is the height of their lowest common ancestor (LCA):

$$d_C(a,b) = h(\mathrm{lca}(a,b))$$

Theorem (Cophenetic ultrametric): Cophenetic distance satisfies the strong triangle inequality:

$$d_C(a,c) \leq \max(d_C(a,b), d_C(b,c))$$

Theorem (Triadic rigidity): For any three leaves, the two largest pairwise cophenetic distances are equal. `[DOI: 10.5281/zenodo.20213043]`

The cophenetic distance IS the ultrametric distance on the Bruhat–Tits tree. The distinction between "Bruhat–Tits trees" and "cophenetic trees" is a framing choice — they are the same mathematical structure. The tree is Bruhat–Tits; the distance function is cophenetic. `[LLM-INFERRED — see q-pna/[source file] §4 for full argument]`

2.3 p-Adic Valuation Encoding

`[ARCHIVE: ultrametric-ai-poc]`

The bridge from tokens to tree leaves is the p-adic valuation. Let $\Sigma = p_1, p_2, \ldots, p_k$ be a set of distinct primes called **semantic primes**. Each semantic prime represents a fundamental dimension of meaning.

Definition (Prime product): For token t with semantic prime strengths $f_i(t) \in \mathbb{N}$:

$$P(t) = \prod_{i=1}^{k} p_i^{f_i(t)}$$

Definition (p-adic valuation vector):

$$\vec{v}(t) = (v_{p_1}(P(t)), v_{p_2}(P(t)), \ldots, v_{p_k}(P(t)))$$

where $v_{p_i}(n)$ is the exponent of p_i in the prime factorization of n.

Definition (Ultrametric distance on valuation vectors):

$$d_{\text{ultra}}(t_1, t_2) = \max_i |v_{p_i}(P(t_1)) - v_{p_i}(P(t_2))|$$

This is the Chebyshev distance on valuation vectors, which inherits ultrametricity from the p-adic valuation.

Demonstration (working code): The `ultrametric-ai-poc` uses $\Sigma = 2, 3, 5, 7, 11$ mapped to semantic categories $\text{good}, \text{bad}, \text{not}, \text{very}, \text{but}$ via WordNet hypernym paths. Each English word is assigned semantic primes, the prime product is computed, and the valuation vector is used for ultrametric attention. Cocycle verification (strong triangle inequality) passes on all token triplets `[ARCHIVE: ultrametric-ai-poc]`.

2.4 Distinction Calculus (Spencer-Brown)

`[ARCHIVE: Auditable Attention PoC]`

The syntactic token calculus is built on Spencer-Brown's Laws of Form (1969) primitives:

Primitive	Notation	Meaning
Mark	`#`	The act of drawing a distinction
Enclosure	`[A]`	A boundary containing expression A
Void	(blank)	The absence of any distinction — not a symbol

Two axioms (rewrite rules):

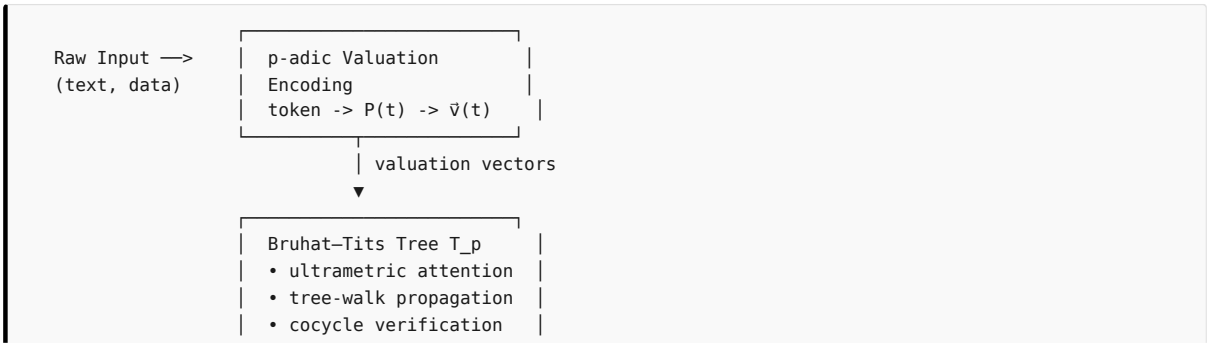
1. **Calling (Idempotence):** ## → # — adjacent marks condense into one. Repetition of the same distinction is idle. Corresponds to $A \wedge A = A$.
2. **Crossing (Involution):** `[[A]]` → A — an enclosure containing only another enclosure cancels. To cross a boundary twice is to uncross it. Corresponds to $\neg\neg A = A$.

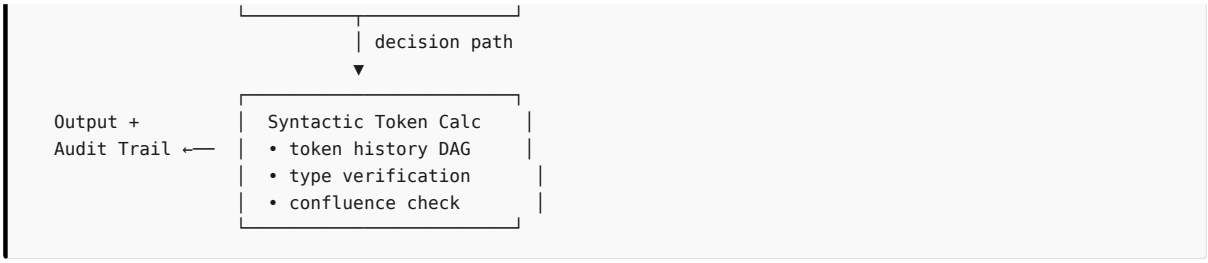
Key insight (Q5 from `Ultrametric Intelligence/[source file]`): The Bruhat–Tits tree $\mathcal{T}_p$ is isomorphic to the set of all finite bracket expressions under Spencer-Brown's calculus. Every token path in the tree is a normal form of some distinction expression. The LCA of two leaves is the longest common prefix of their normal forms. This means the distinction calculus IS the formal verification substrate for operations on the Bruhat–Tits tree.

---

3. Architecture

3.1 Overview





3.2 Token Encoding

Step 1 — Semantic prime assignment: Map each input token t to a set of semantic primes with strengths. For text: WordNet hypernym paths determine which semantic categories apply (as in `ultrametric-ai-poc`). For structured data: feature hierarchy determines the encoding levels. For unstructured data (images, audio): hierarchical clustering on training data determines the prime assignment ``.

Step 2 — Prime product computation:

$$P(t) = \prod_{i=1}^{k} p_i^{f_i(t)}$$

where $f_i(t) \in \mathbb{N}$ is the strength of semantic prime p_i. A larger exponent means stronger association with that semantic dimension.

Step 3 — Valuation vector extraction:

$$\vec{v}(t) = (v_{p_1}(P(t)), v_{p_2}(P(t)), \ldots, v_{p_k}(P(t)))$$

Each component $v_{p_i}(P(t)) = f_i(t)$ — the exponent is exactly the strength. This yields a vector in $\mathbb{N}^k$.

Step 4 — Leaf activation mapping: The valuation vector $\vec{v}(t)$ for a single token does NOT directly map to a single leaf. Instead, the encoding for an entire input sequence produces a distributed **leaf activation vector** $\mathbf{a} = (a_1, \ldots, a_L) \in [0,1]^L$ across all L leaves. The activation a_ℓ at leaf ℓ is determined by the ultrametric attention mechanism (§3.3): tokens attend to each other based on tree distances, and the resulting attended valuations are distributed across leaves whose positions correspond to the semantic prime combinations present in the input.

Concrete mapping (from `ultrametric-ai-poc`): In the working PoC, each unique valuation vector $\vec{v}(t)$ corresponds to a leaf whose path from root is determined by the sequence of valuation component values. Tokens with identical valuation vectors map to the same leaf. The leaf activation a_ℓ is the aggregated attention weight from all tokens whose valuation vectors map to leaf ℓ. This ensures that the leaf activation vector $\mathbf{a}$ inherits the ultrametric structure of the tree — similar tokens produce activations at nearby leaves.

Encoding properties: - **Ultrametric:** Distance between any two encodings satisfies the strong triangle inequality - **Hierarchical:** Similar tokens (sharing semantic primes) produce activations at nearby leaves - **Discrete:** The encoding space is a finite set of L leaves, not a continuous manifold - **Interpretable:** Each dimension of $\vec{v}(t)$ is a named semantic category; each leaf corresponds to a specific combination of categories

3.3 Ultrametric Attention

`[ARCHIVE: ultrametric-ai-poc]`

Attention weights on the Bruhat–Tits tree are computed from ultrametric distances between token encodings. **No learned parameters** — the attention pattern emerges purely from tree geometry.

Algorithm:

Input: Sequence of tokens $t_1, \ldots, t_n$, temperature $T > 0$
Output: Attention matrix $A \in \mathbb{R}^{n \times n}$, attended representations $\vec{v}_i^{\text{out}}$

1. Encode tokens: $\vec{v}_i = \vec{v}(t_i)$ for $i = 1, \ldots, n$
2. Compute pairwise ultrametric distances:

$$d_{ij} = d_{\text{ultra}}(t_i, t_j) = \max_k |v_{p_k}(P(t_i)) - v_{p_k}(P(t_j))|$$

3. Compute attention weights (exponential decay):

$$A_{ij} = \frac{\exp(-d_{ij}/T)}{\sum_{\ell=1}^{n} \exp(-d_{i\ell}/T)}$$

4. Compute attended output: $$\vec{v}^{\text{out}}*i = |sum*j$$}^{n} A_{ij} \cdot \vec{v

Properties:

Property	Standard Attention (Dot-Product)	Ultrametric Attention
Parameter count	$O(d^2)$ (query, key, value matrices)	0 (no learned parameters)
Similarity	$\langle q_i, k_j \rangle / \sqrt{d}$	$\exp(-d_{\text{ultra}}(t_i, t_j)/T)$
Interpretability	Opaque — learned projections	Transparent — distance = LCA depth
Gauge invariance	No — rotation changes attention	Yes — ultrametric distance is invariant under tree automorphisms
Auditability	No traceable explanation	Every weight traces to a specific LCA

Cocycle condition: For any three tokens t_i, t_j, t_k, the ultrametric distances must satisfy:

$$d_{ik} \leq \max(d_{ij}, d_{jk})$$

This is computationally verified at inference time. Violations indicate inconsistency in the cognitive representation `[ARCHIVE: ultrametric-ai-poc]`.

Temperature interpretation: The temperature T controls attention sharpness: - $T \to 0$: Hard attention — only tokens at identical leaves attend to each other - $T \to \infty$: Uniform attention — all tokens attend equally - $T \approx 1$: Soft attention — tokens at nearby leaves (deep LCA) attend strongly

3.4 Output Decoding — Decision Paths

A Q-PNA output is not a probability distribution over class labels. It is a **decision path** — a traceable sequence of nodes from root to leaf:

$$\text{Path} = v_0 \xrightarrow{c_1} v_1 \xrightarrow{c_2} v_2 \xrightarrow{c_3} \cdots \xrightarrow{c_D} v_D = \text{leaf}$$

where v_0 is the root, each v_i is a node at depth i, and $c_i \in 0, 1, \ldots, p$ is the child index followed.

Decoding procedure: 1. Start at root v_0 with the attended representation of the input 2. At each internal node v, compute the activation of each child c as the sum of attention-weighted valuations in that subtree 3. Select the child with maximal activation (greedy) or sample proportionally (stochastic, with temperature) 4. Recurse until a leaf is reached 5. The leaf's associated label/action is the output. The path from root to leaf IS the audit trail.

The decision path IS the explanation. No post-hoc interpretability method (LIME, SHAP) is needed. Every decision is a geometric path whose meaning is grounded in the tree structure — the shared LCA relationships with training examples, the semantic primes activated at each branch, and the token history that produced the leaf activation.

From attention to leaf activations: The attended valuation vectors $\vec{v}^{\text{out}}*i $produced by ultrametric attention (§3.3) are aggregated into a leaf activation vector |mathbf{a} \in [0,1]^L as follows: each valuation vector \vec{v}^{|text{out}}*i is mapped to its corresponding leaf |ell(i) in |mathcal{T}_p, and the leaf activation is the sum of attention weights directed to that leaf: a|ell = |sum |sum_j A^* Extra close brace or missing open brace. This aggregated leaf activation vector is what the cophenetic loss (§4) compares against the target encoding $\mathbf{a

4. Loss Function on Ultrametric Trees

4.1 Cophenetic Loss

`[QWAV-INTERNAL: Q-PNA v0.1 specification]`

For a training example with input encoding $\mathbf{a} = (a_1, \ldots, a_L)$ (leaf activations) and target encoding $\mathbf{a}^* = (a_1^*, \ldots, a_L^*)$, the **expected cophenetic distance** is:

$$D_C(\mathbf{a}, \mathbf{a}^*) = \sum_{i=1}^{L} \sum_{j=1}^{L} a_i a_j^* \cdot d_C(i, j)$$

where $d_C(i, j) = h(\mathrm{lca}(\mathrm{leaf}_i, \mathrm{leaf}_j))$ is the cophenetic distance between leaves i and j.

Cophenetic loss:

$$\mathcal{L}_{\text{coph}}(\mathbf{a}, \mathbf{a}^*) = D_C(\mathbf{a}, \mathbf{a}^*)$$

Intuition: If the predicted activation and target activation are concentrated on leaves that share a deep LCA (close in the tree), the loss is small. If they're on distant branches (shallow LCA), the loss is large. This is naturally hierarchical — errors at coarser levels of the hierarchy are penalized more heavily.

4.2 Multi-Resolution Decomposition

The cophenetic loss can be decomposed by tree level:

$$\mathcal{L}_{\text{MR}} = \sum_{\ell} w_\ell \cdot \mathcal{L}

where $w_\ell = 2^{-\ell}$ (emphasizing coarser levels) and $\mathcal{L}_\ell$ measures error at depth ℓ:

$$\mathcal{L}_\ell = \sum_{v \in \text{nodes}(\ell)} \left| \mathbf{a}_v - \mathbf{a}

with $\mathbf{a}_v = \sum a_i$ being the aggregated activation in the subtree rooted at v.(v)

4.3 Properties

Property	Cross-Entropy (Standard)	Cophenetic Loss (Q-PNA)
Space	Continuous $\mathbb{R}^n$	Discrete tree $\mathcal{T}_p$
Hierarchy awareness	None — all output dimensions are independent	Errors weighted by tree distance — hierarchical structure respected
Decomposability	Single scalar	Decomposable by tree level (multi-resolution)
Formal verifiability	Approximate (gradients)	Exact — the error at each node has geometric meaning
Triadic rigidity	Not applicable	Inherent: for any three examples, two loss pairings are equal

`` The cophenetic loss is defined but has not been computationally validated as a training objective. Convergence properties, relationship to classification accuracy, and comparison to cross-entropy on standard benchmarks are open questions. The definition is provided as the specification — validation is Phase 0 below.

---

5. Optimization: Tree-Walk Algorithm

5.1 Why Gradient Descent Doesn't Apply

Gradient descent requires a differentiable manifold. The Bruhat–Tits tree is a discrete metric space — derivatives are not defined. Q-PNA replaces gradient-based optimization with **tree-walk optimization**: a discrete search algorithm that redistributes edge weights by decomposing error through the tree hierarchy.

`[QWAV-INTERNAL: Q-PNA v0.1 specification]`

5.2 Algorithm

Input: Training example (x, y), current tree with edge weights $W = w_{v \to c}$ for all internal nodes v and children c
Output: Updated edge weights W', leaf activations $\mathbf{a}'$
Hyperparameters: Learning rate $\eta > 0$, convergence threshold $\varepsilon > 0$, temperature $T > 0$

Algorithm (one training step):

```
1. FORWARD PASS:
   a. Encode input x $\to$ leaf activations a
   b. For each internal node v (bottom-up):
      For each child c of v:
          Aggregate activation: A_c = sum(attended activations in subtree(c))
      Normalize across children: A_c = A_c / sum(A_*)
   c. Decode output: greedy path from root $\to$ leaf $\to$ output activations a^out

2. COMPUTE LOSS:
   L = L_coph(a^out, a^*)   # cophenetic distance to target

3. ERROR DECOMPOSITION (top-down):
   For each internal node v at depth ℓ:
       E_v = sum_{i in leaves(v)} |a^out_i - a^*_i|
       # This is the total error originating in subtree v

4. EDGE WEIGHT UPDATE (discrete gradient analog):
   For each internal node v:
       E_bar_v = mean(E_c for each child c of v)
       For each child c of v:
           w_{v$\to$c} ← w_{v$\to$c} - η · (E_c - E_bar_v)
       # Redistribute weight away from high-error subtrees
       # Normalize: sum(w_{v$\to$*}) = 1

5. LEAF ACTIVATION REFINEMENT:
   For each leaf i where |a^out_i - a^*_i| > τ:
       a_i ← a_i + η · (a^*_i - a_i)
       # Move leaf activation toward target for large errors

6. REPEAT steps 1–5 for T iterations or until L < ε
```

5.3 Relationship to Backpropagation

Backpropagation	Tree-Walk Optimization
Gradient $\partial \mathcal{L} / \partial w$	Subtree error E_v
Chain rule through layers	Tree-walk from root to leaves (top-down decomposition)
Continuous weight update	Discrete weight redistribution
Vanishing gradients in deep networks	Error naturally attenuates with tree depth (strong triangle inequality: errors at deep nodes are contained within their subtree)
Requires differentiable activations	Works on any activation — only distances and aggregations needed

5.4 Convergence Conditions

`` The following convergence conditions are hypothesized but not proven:

1. **Monotonicity:** $\mathcal{L}_{\text{coph}}$ should decrease monotonically if η is sufficiently small. The error propagation through the tree via E_v is conservative (total error is preserved, only redistributed). This is the discrete analog of the gradient being a descent direction.
2. **Learning rate bound:** η must be bounded by the minimum edge weight divided by the maximum subtree error difference. Too large η causes weight oscillation (weights flip between extreme values).

3. **Tree depth stability:** Deeper trees ($D > 10$) may suffer from error dilution — the error signal at deep nodes becomes small relative to the total error, slowing convergence. This is analogous to vanishing gradients but structurally different: the error is diluted spatially (across many subtrees) rather than multiplicatively.
4. **Convexity analog:** The cophenetic loss is not convex in the standard sense (the domain is discrete). However, the loss landscape on the tree may exhibit a property analogous to geodesic convexity in ultrametric spaces.

5.5 Computational Complexity

For a truncated tree $\mathcal{T}_p$ of depth D with $L = (p+1) \cdot p^{D-1}$ leaves:

Operation	Complexity	Notes
Encoding (per token)	$O(k \cdot \log M)$	k semantic primes, M max product value
Ultrametric attention (n tokens)	$O(n^2 \cdot k)$	Pairwise distances, k valuation dimensions
Forward pass (tree propagation)	$O(L)$	Each leaf aggregated once, bottom-up
Error decomposition	$O(L)$	Each node visited once, top-down
Edge weight update	$O(L)$	Each edge updated once
Per training step	$O(n^2 k + L)$	Quadratic in sequence length, linear in tree size
Standard transformer (comparison)	$O(n^2 d)$	Quadratic in sequence length, d = embedding dimension

For near-term feasibility: $p = 3$, $D = 5$ gives 324 leaves — computationally tractable on a laptop. $D = 10$ gives ~80,000 leaves — still feasible for batch processing. The dominant cost is not the tree but the $O(n^2 k)$ pairwise attention.

5.6 Quantum Walk Speedup

`[LLM-INFERRED — aspirational, not validated]`

When implemented on quantum hardware, the forward pass benefits from quantum walk speedup: $O(\sqrt{n})$ time to traverse the tree vs. $O(n)$ classically. This is the primary quantum advantage — not in the optimization step, but in the inference (forward pass) step. The quantum walk is a continuous-time quantum walk on the tree graph, which can reach target leaves quadratically faster than classical random walks.

Honest note: This depends on fault-tolerant quantum hardware not currently available. The classical simulation path (tree-walk on a CPU/GPU) is the near-term implementation strategy.

---

6. Syntactic Token Calculus

6.1 Purpose

`[ARCHIVE: Syntactic Token Calculus v2/v3]`

The Syntactic Token Calculus (STC) is a formal framework for reasoning about computations on the Bruhat–Tits tree. It provides:

1. **Deterministic confluence:** Every computation has a unique, traceable path through the tree
2. **Formal verification:** Decisions can be proven correct (or incorrect) by examining the token transformations along the path
3. **Elimination of singularities:** By treating reality as discrete topological enclosures (tokens), the calculus avoids singularities that plague continuous models
4. **Gauge-invariant meaning:** Cross-ratio invariants define meaning independent of tree automorphisms `[ARCHIVE: Syntactic Token Calculus v3.1]`

6.2 Token Definition

A **token** τ is a discrete topological enclosure — a labeled node in the Bruhat–Tits tree with associated data:

$$\tau = (v, \text{type}, \text{data}, \text{parent})$$

Component	Type	Description
v	$\mathcal{T}_p$ node	Position in the tree (depth, path from root)
type	$\mathcal{T}$ (finite type system)	The token's type — determines valid operations
data	Discrete payload	The token's information content (integer, string, or nested tokens)
parent	Token reference or $\varnothing$	The parent token — $\varnothing$ for root tokens

6.3 Token Operations

All computation in STC is expressed as token operations on the tree:

Operation	Symbol	Description	Type Constraint
Spawn	$\tau \to \tau_1, \ldots, \tau_{p+1}$	A token splits into child tokens at depth $\ell + 1$	Children inherit parent type
Merge	$\tau_1, \ldots, \tau_{p+1} \to \tau$	Child tokens combine into a parent token at depth ℓ	All children must have compatible types
Transform	$\tau \xrightarrow{f} \tau'$	Token data is transformed by function f	Type and position unchanged
Move	$\tau \xrightarrow{v \to v'} \tau'$	Token moves to adjacent node (parent, child, or sibling)	Must respect tree adjacency
Annihilate	$\tau \to \varnothing$	Token is removed	Parent reference must be updated

Type compatibility for Merge: Two types T_1 and T_2 are compatible if they share a common supertype in the type lattice. The merged token receives the least upper bound type: $\text{lub}(T_1, T_2)$.

6.4 Computation as Token History

A computation is a **token history** — a directed acyclic graph (DAG) of token operations, where each edge represents an operation and each node represents a token state:

$$H = (\tau_t, \text{op}_e)$$

Properties: - **Causality:** Every operation has well-defined inputs and outputs - **Traceability:** Every output token can be traced back to its originating input tokens - **Deterministic confluence:** For any two paths through H that reach the same token, the resulting token state is identical

6.5 Verification Protocol

To verify that output y is correct for input x:

1. **Reconstruct the token history** H from the Q-PNA's decision trace
2. **Check type consistency:** Every token operation respects the type system — spawn preserves types, merge requires compatible types, transform preserves type
3. **Check path validity:** Every Move operation respects the tree structure — tokens may only move to parent, child, or sibling nodes
4. **Check confluence:** For any two paths through H that reach the same token, the resulting token state is identical (determinism)
5. **Check against specification:** The final output token's data matches the specification for input x

Result: If all checks pass, the computation is **provably correct** — the decision path IS the proof.

6.6 Relationship to Distinction Calculus

`[ARCHIVE: Auditable Attention PoC]`

The STC operations can be expressed as Spencer-Brown distinction calculus operations:

STC Operation	Distinction Calculus	Meaning
Spawn	$\tau \to \tau_1, \ldots, \tau_{p+1}$	Enclose: $\tau \to [\tau_1 \tau_2 \ldots \tau_{p+1}]$
Merge	$\tau_1, \ldots, \tau_{p+1} \to \tau$	Cross: $[[\tau_1 \ldots \tau_{p+1}]] \to \text{reduced form}$
Transform	$\tau \xrightarrow{f} \tau'$	Calling: repeated marks condense, or a function application
Annihilate	$\tau \to \varnothing$	Void: the token returns to the unmarked state
Move	$\tau \xrightarrow{v \to v'} \tau'$	Re-entry: the token appears at a different structural position

This connection means that every STC computation has a purely syntactic interpretation — it can be expressed without real numbers, only marks and enclosures. The distinction calculus is the formal verification substrate.

---

7. Glass-Box Verification Protocol

7.1 Glass-Box Definition

A Q-PNA model is **glass-box** (as opposed to black-box) if it satisfies all five conditions:

1. **Path traceability:** Every output includes the full decision path from root to leaf
2. **Operation auditability:** Every token operation along the path is logged and inspectable
3. **Type consistency:** All operations respect the STC type system
4. **Deterministic confluence:** The same input always produces the same decision path (up to stochastic sampling, where the sampling distribution is logged)
5. **Cocycle satisfaction:** The strong triangle inequality holds for all token triplets encountered

7.2 Audit Trail Format

Every Q-PNA decision produces an audit trail:

```
Decision: [output label]
Path: root $\to$ node[2] $\to$ node[2,1] $\to$ node[2,1,3] $\to$ leaf[47]

Token operations:
  [spawn]  root $\to$ {τ_1, τ_2, τ_3, τ_4}
  [transform] τ_2 $\to$ f_embed(τ_2)
  [move]   τ_2 $\to$ child[2,1]
  [merge]  {τ_2, τ_aux} $\to$ τ_out
  [transform] τ_out $\to$ classifier(τ_out) = [output label]

Tree context:
  LCA(τ_2, τ_aux) = node[2] at depth 2
  d_ultra(τ_2, τ_aux) = 2

Verification:
  type-consistent ✓  |  path-valid ✓  |  confluent ✓  |  cocycle ✓
```

This audit trail is **human-readable** (a regulator can understand the decision) AND **machine-verifiable** (automated checks pass/fail each condition).

7.3 Comparison: Black-Box vs. Glass-Box

Property	Black-Box AI (Standard DNN)	Glass-Box AI (Q-PNA)
Decision path	Opaque composition of nonlinearities	Geometric path root $\to$ leaf
Audit trail	Approximate (LIME, SHAP, attention viz)	Exact — the path is the explanation
Formal verification	Impossible in general	Possible via token calculus type checking
Regulatory compliance	Requires external auditing	Self-documenting by construction
Error attribution	Statistical (which input features?)	Structural (which subtree?)
Bias detection	Post-hoc statistical analysis	Pre-computation: biased paths visible in tree structure
Adversarial robustness	Fragile to ε-perturbations	Gauge-invariant — no continuous corridors to exploit `[ARCHIVE: Auditable Attention PoC v0.2.1]`

7.4 Cocycle Condition as Consistency Check

`[ARCHIVE: ultrametric-ai-poc]`

The cocycle condition is a runtime verification that the cognitive representation is internally consistent. For any three tokens t_i, t_j, t_k:

$$d_{\text{ultra}}(t_i, t_k) \leq \max(d_{\text{ultra}}(t_i, t_j), d_{\text{ultra}}(t_j, t_k))$$

If this fails for any triple, the representation contains an inconsistency — the tokens cannot be simultaneously embedded in an ultrametric space. The violation is logged and flagged for investigation.

Interpretation: The cocycle condition is to cognitive consistency what conservation laws are to physics — a structural invariant that must hold. A violation indicates that the semantic prime assignment or the token encoding has produced a contradictory representation.

---

8. Relationship to Quantum Architecture

8.1 Shared Mathematical Foundation

`[QWAV-INTERNAL: Technical Deep-Dive §4]`

Both Q-PNA (AI) and UQC (Quantum Computing) operate on the same mathematical structure — the Bruhat–Tits tree $\mathcal{T}_p$ — but exploit different properties:

Property	UQC (Quantum)	Q-PNA (AI)
Primary use of tree	Error confinement via strong triangle inequality	Hierarchical feature organization
Encoding	q-ary scatter across leaves	p-adic valuation vectors
Propagation	Holographic tensor network (perfect tensors)	Classical tree-walk (quantum walk optionally)
Key mechanism	Passive fault tolerance — no active QEC needed	Glass-box explainability by construction
Verification	Logical error rate (LER) simulation	Token calculus formal verification

Property	UQC (Quantum)	Q-PNA (AI)
Hardware	40-atom neutral atom at 4 K (validated computationally)	Room temperature (classical simulation)

8.2 The Common Thesis

> Continuous manifolds are the wrong mathematical foundation for computation. Ultrametric (tree-based) geometry provides structural properties — error confinement for quantum, hierarchical transparency for AI — that are unavailable in Archimedean spaces.

8.3 Complementary Roles

- **Quantum side** provides the hardware pathway: 40-atom neutral atom implementation at 4 K, passive fault tolerance, q-ary scatter amplification, with computational validation published (Tier 0 + Tier 1 papers, DOI: 10.5281/zenodo.20134944, 10.5281/zenodo.20208437) ``
- **AI side** provides the software pathway: glass-box decision-making, formal verifiability, token calculus for regulatory compliance — specified here, not yet validated
- **Together:** A complete computing architecture — hardware AND software — built on a single mathematical correction to the Archimedean assumption

---

9. Computational Validation Plan

`` The following phases are specified but not yet executed. This is a research roadmap, not a progress report.

9.1 Phase 0: Tree-Walk Optimization Simulation

Goal: Demonstrate that tree-walk optimization converges on a toy hierarchical classification task.

Setup: - Tree: $p = 3$, depth $D = 5$ (324 leaves) - Task: Synthetic hierarchical classification (e.g., 3-level taxonomy with 27 classes) - Encoding: p-adic valuation vectors with $k = 5$ semantic primes - Metrics: Cophenetic loss over training epochs, classification accuracy, audit trail completeness

Success criterion: Loss decreases monotonically over 100 epochs on synthetic data. Classification accuracy exceeds random baseline by significant margin.

9.2 Phase 1: Token Calculus Implementation

Goal: Implement the STC type system and verification protocol in Python.

Setup: - Define token types, operations, and verification rules - Generate token histories from simulated Q-PNA decisions - Verify type consistency, path validity, and confluence

Success criterion: 100% of valid token histories pass verification. 100% of intentionally corrupted histories (type mismatch, invalid move, non-confluent paths) fail verification.

9.3 Phase 2: Integration with Quantum Simulation

Goal: Demonstrate that the same tree topology used for quantum error confinement can encode hierarchical features for AI classification — proving the shared-geometry thesis.

Setup: - Use the `ultrametric_v2` codebase's Bruhat–Tits tree implementation `[QWAV-INTERNAL: ultrametric_v2 codebase]` - Encode a hierarchical dataset on the SAME tree used for quantum error simulation ($p = 3$, $D = 7$) - Run both quantum error simulation AND AI classification on the same tree

Success criterion: The tree simultaneously suppresses quantum errors AND organizes hierarchical features. (This is the strongest form of the common thesis.)

9.4 Phase 3: Comparative Benchmarking

Goal: Compare Q-PNA against standard architectures on glass-box metrics.

Metrics: - Audit trail completeness (% of decisions with full trace) - Verification success rate (% of outputs provably correct via STC) - Decision path length vs. model depth - Training convergence (cophenetic loss vs. cross-entropy) - Adversarial robustness (accuracy under perturbation, compared to equivalent-capacity MLP)

---

10. Honest Limitations & Open Questions

10.1 What This Specification Does NOT Claim

- ❌ Q-PNA has not been implemented or computationally validated as a full architecture
- ❌ No empirical results exist for tree-walk optimization convergence
- ❌ Quantum walk speedup assumes fault-tolerant quantum hardware not yet available
- ❌ Token calculus has not been formalized in a proof assistant (Lean/Coq)
- ❌ The AI side is less developed than the quantum side — this specification is a starting point, not a finished product
- ❌ No comparison to transformers, GNNs, or other architectures on standard benchmarks

10.2 What IS Validated

- ✅ Ultrametric attention with p-adic valuation encoding: demonstrated in working Streamlit app (`ultrametric-ai-poc`)
- ✅ Cophenetic distance ultrametric inequality: proven and computationally verified (`Tree Distance Cophenetic.md`)
- ✅ Syntactic token calculus: book-length formal treatment exists (STC v2, v3)
- ✅ Distinction calculus primitives: formalized (Spencer-Brown 1969, STC [source file])
- ✅ Cocycle verification: implemented and tested (`cocycle.py`)
- ✅ Quantum side: two published papers with computational validation of error confinement

10.3 Open Research Questions

1. **Convergence guarantees:** Under what conditions does tree-walk optimization converge? Is there a discrete analog of the universal approximation theorem?
2. **Scalability:** What is the computational complexity of tree-walk optimization as a function of tree depth D and branching factor $p + 1$? Can it scale to ImageNet-size problems?
3. **Quantum advantage threshold:** At what tree size does quantum walk speedup become practically significant? What is the crossover point vs. classical simulation?
4. **Expressiveness:** What function classes can Q-PNA represent? Is the tree structure a limitation or a feature?
5. **Learned hierarchies:** For unstructured data (images, raw audio), how should the semantic prime assignment be learned? Is hierarchical clustering sufficient, or is end-to-end learning required?
6. **Token calculus completeness:** Is the STC type system expressive enough for general computation, or is it restricted to a specific class of problems?
7. **Hybrid architectures:** Can Q-PNA be combined with standard neural components? E.g., CNN for feature extraction $\rightarrow$ Q-PNA for classification with glass-box audit trail?
8. **Distinction calculus bridge:** The five open questions from `Ultrametric Intelligence/[source file]` remain open — the formal bridge between Spencer-Brown primitives and ultrametric attention has not been completed.
9. **Batch semantics:** The STC assumes single-token operations. Batch computation semantics for training efficiency have not been defined.

10. **Optimal semantic prime selection:** The current semantic prime set $2, 3, 5, 7, 11$ is heuristic. Is there a principled method for selecting semantic primes from data?

10.4 What Closes These Questions

- Computational validation (Phases 0–3 in §9)
- Formal analysis of the tree-walk optimization algorithm
- Comparative benchmarking against standard architectures
- Reader review and critique from the AI/ML community
- Formalization of the token calculus in Lean 4 or Coq

---

11. Comparison to Standard Architectures

11.1 Transformers

Dimension	Transformer	Q-PNA
Representation space	$\mathbb{R}^d$ (continuous)	Bruhat–Tits tree $\mathcal{T}_p$ (discrete)
Attention	Learned Q, K, V projections ($O(d^2)$ params)	Ultrametric distance decay (0 learned params)
Position encoding	Sinusoidal or learned	Inherent in tree position
Interpretability	Attention visualization (post-hoc)	Decision path (inherent)
Training	Gradient descent via backprop	Tree-walk optimization (discrete)
Adversarial robustness	Fragile	Gauge-invariant embedding
Scaling	Demonstrated at 100B+ params	Not yet demonstrated

11.2 Graph Neural Networks (GNNs)

Dimension	GNN	Q-PNA
Graph structure	Input-dependent (arbitrary graph)	Fixed tree $\mathcal{T}_p$
Message passing	Learned aggregations over neighbors	Ultrametric attention over tree paths
Hierarchy	Must be learned or provided	Inherent in tree structure
Explainability	GNNExplainer, post-hoc	Decision path IS the explanation
Formal verification	Not available	Token calculus verification protocol

11.3 Decision Trees / Random Forests

Dimension	Decision Tree	Q-PNA
Tree structure	Learned from data (greedy splits)	Fixed Bruhat–Tits tree (mathematically structured)
Decision	Single path (hard)	Soft path (attention-weighted)
Interpretability	High (if shallow) — path is readable	High — path is readable + formally verifiable
Expressiveness	Limited (axis-aligned splits)	Potentially richer (ultrametric combinations)

Dimension	Decision Tree	Q-PNA
Training	Greedy recursive partitioning	Tree-walk optimization (global error redistribution)

11.4 Where Q-PNA Could Excel

Q-PNA is not proposed as a universal replacement for all neural architectures. Its advantages are specific:

1. **Regulated domains** (healthcare, finance, law): Where decisions must be explainable and auditable. The glass-box audit trail meets regulatory requirements that black-box models cannot.
2. **Hierarchical data** (taxonomies, ontologies, biological classification): Where the tree structure naturally matches the data structure. The cophenetic loss directly optimizes hierarchical accuracy.
3. **Adversarial environments** (security, fraud detection): Where gauge invariance provides inherent robustness to perturbation-based attacks.
4. **Formal verification requirements** (safety-critical systems): Where the token calculus enables machine-verifiable correctness proofs.
5. **Quantum-classical hybrid systems** (future): Where the shared tree geometry with QWAV's quantum architecture enables a unified computing stack.

---

12. Prior Work Attribution

This specification synthesizes the following prior work:

Work	Location	Contribution
Ultrametric AI PoC	`[ARCHIVE] ultrametric-ai-poc`	Working demonstration: p-adic valuation encoding, ultrametric attention, cocycle verification, distinction calculus integration
Auditable Attention PoC	`[ARCHIVE] Auditable Attention PoC`	Technical narrative: gauge-invariance framing, STC v3.1 foundations, Spencer-Brown primitives
Ultrametric Intelligence	`[ARCHIVE] Ultrametric Intelligence`	Synthesis: non-Archimedean geometry + AI. Five open questions bridging distinction calculus and ultrametric attention
STC v2/v3	`[ARCHIVE] Syntactic Token Calculus v2/v3`	Book-length formal treatment of token calculus — type system, operations, verification
Tree Distance Cophenetic	`[DOI: 10.5281/zenodo.20213043]`	Mathematical foundation: cophenetic distance, triadic rigidity, ultrametric inequality proofs
Language as Information Architecture	`[DOI: 10.5281/zenodo.20137616]`	Empirical linguistics: token encoding theory, mutual exclusion
PANN	`[ARCHIVE] PANN (github.com/rwnq8/PANN)`	Prior art: p-adic attention in PyTorch, hierarchical loss functions, topological regularization
QWAV Quantum Papers	`[Zenodo] (https://zenodo.org/communities/qwav/)`	Quantum side validation: error confinement at $p = 3$, $D = 7$, zero logical errors
QWAV v0.1 Spec	`[QWAV-INTERNAL] Q-PNA v0.1`	Original Q-PNA specification — starting point for this document

---

Data Availability

All archival project files referenced in this specification are publicly available through Google Drive:

QWAV Public Archive

All code, specifications, and computational results for this project are available at the public repository: **github.com/QNFO/Q-PNA**

This archive includes: ultrametric-ai-poc (working Streamlit demonstration), Proof-of-Concept for Auditable Attention using Ultrametric Tree Distances, Ultrametric Intelligence, Syntactic Token Calculus v2 and v3, Formal Ontology of Distinction and Invariance, and related projects. PANN is available at github.com/rwnq8/PANN.

Published works with DOIs are available through Zenodo at the links provided. QWAV internal working documents (`QWAV-INTERNAL`) are available upon request.

References

1. Quni-Gudzinas, R. B. (2026). *The Tree Distance Cophenetic: A Unified Framework for Hierarchical Ontology.* Zenodo. DOI: 10.5281/zenodo.20213043
2. Quni-Gudzinas, R. B. (2026). *Language as Information Architecture.* Zenodo. DOI: 10.5281/zenodo.20137616
3. Quni-Gudzinas, R. B. (2026). *Computational Validation of Ultrametric Error Confinement in Bruhat–Tits Tree Quantum Circuits.* Zenodo. DOI: 10.5281/zenodo.20134944
4. Quni-Gudzinas, R. B. (2026). *Symmetric Extension of Ultrametric Error Confinement.* Zenodo. DOI: 10.5281/zenodo.20208437
5. Spencer-Brown, G. (1969). *Laws of Form.* George Allen and Unwin.
6. [ARCHIVE] ultrametric-ai-poc — Ultrametric AI Proof of Concept. Public archive: Google Drive
7. [ARCHIVE] Proof-of-Concept for Auditable Attention using Ultrametric Tree Distances. Public archive.
8. [ARCHIVE] Ultrametric Intelligence — Synthesis of Non-Archimedean Geometry and AI. Public archive.
9. [ARCHIVE] Syntactic Token Calculus v2 and v3. Public archive.
10. [ARCHIVE] PANN — Prime-Attentive Neural Networks. github.com/rwnq8/PANN
11. [QWAV-INTERNAL] Q-PNA v0.1 Specification. Available upon request.
12. [QWAV-INTERNAL] Technical Deep-Dive — Ultrametric Quantum Computing and AI. Available upon request.